UNO Java Swing Project – Final Product Backlog

Project: UNO Card Game

Course: Programming (2IP90), CBL Java Application Development

Platform: Java + Swing, with an additional terminal interface

1. Project Idea

A playable UNO game for one human player against a computer opponent. The Swing interface allows the player to play cards, draw cards, choose wild-card colours, and save or load a game. The application maintains hands, piles, turns, direction, and draw penalties, gives feedback, and announces a winner when a hand becomes empty.

The project separates model, logic, controller, view, bot, persistence, and utility packages. Both interfaces share the same game engine. The computer-opponent scope includes a Minimax strategy and a basic strategy for comparison.

2. Advanced Technical Topics

Advanced Topic 1 – Game-State Persistence, Serialization, and Compression

Collect the game data in SaveState, serialize it into versioned UTF-8 bytes with GameSerializer, and compress it using the custom Compressor. SaveManager writes the result to a .unosave file. Loading reverses these operations and rebuilds the game objects.

The compression algorithm uses literal bytes and references to repeated earlier sequences. References contain a 12-bit distance and a 4-bit encoded length. The format includes a UZ signature, original byte count, and entry flags. Compression is lossless, but does not guarantee a smaller file for every input.

Advanced Topic 2 – Computer Bot Using Minimax

Implement a depth-limited Minimax computer opponent through BotStrategy. It will simulate legal moves, examine opponent responses, and select the move with the best resulting evaluation. The algorithm uses lookahead to evaluate possible consequences before selecting an action.

The work includes isolated state simulation, legal move generation, wild-colour choices, terminal win/loss scoring, a cutoff heuristic, and bounded search. MAX/MIN roles must follow the actual player to move: Skip and Reverse can give the same player another turn.

UNO has hidden hands and uncertain draws. The information model must therefore be explicitly selected and documented: a full-information educational approximation or an approach using sampled possible hidden states. Ordinary perfect-information Minimax must not be presented as a complete solution to hidden-information UNO. The chosen simplifications and their effect on fairness and decisions must be explained.

3. Product Backlog

Priorities: Must = required final scope; Should = valuable improvement; Could = optional extension.

For card-specific demonstrations, prepare a named save fixture with documented hands, top card, turn, and penalty. Use the same fixtures when repeating a demonstration.

Core Game and Architecture

#	Priority	Name	How to Demo	Notes
1	Must	Separate project responsibilities	1. Open the package tree. 2. Identify model, logic, controller, view, bot, persistence, and util. 3. Trace a card click through validation, state change, and display refresh.	GameController coordinates; GameEngine executes rules; GameListener communicates events.
2	Must	Represent cards, hands, and players	1. Inspect number, action, and wild cards in a fixture. 2. Show their colours/values and player name, hand, and bot flag. 3. Add/remove a card in a verification harness and check hand size.	Card, Hand, Player, Color, and Value.
3	Must	Create and shuffle the selected deck	1. Construct a deck in a verification harness. 2. Check 60 cards and documented multiplicities. 3. Compare several shuffled orders.	Deck: 60 cards, with one of each regular colour/value combination and four of each wild type.
4	Must	Start a game and deal hands	1. Launch the GUI. 2. Click Start Game and enter a name. 3. Confirm seven cards each, one non-wild discard, and 45 draw cards. 4. Repeat with a blank name and confirm the default.	Starting discard effects are not executed.
5	Must	Validate ordinary card matching	1. Load a red-five fixture without a penalty. 2. Play a red card or a blue five and confirm acceptance. 3. Reload and try a blue seven; confirm rejection without state change.	RuleValidator matches colour or value.
6	Must	Advance and protect turns	1. Play a regular card. 2. Confirm the bot acts and control returns appropriately. 3. Try human input during the bot turn and confirm no human move occurs.	GameState tracks index and direction; controllers guard input.
7	Must	Draw and end the turn	1. Record human hand size. 2. Click Draw without a pending penalty. 3. Confirm one added card and a turn change.	Drawing ends the turn and is allowed even when another legal move exists.
8	Must	Apply Skip and Reverse	1. Play Skip from a fixture and confirm the opponent loses its turn. 2. Repeat with Reverse. 3. Confirm direction changes and the same player acts again.	Verify both human and bot use; two-player rules.
9	Must	Apply Draw Two and stacking	1. Play +2 against a hand without a response; confirm two draws and a skipped turn. 2. Reload with a stacking card. 3. Stack +2 and confirm a penalty of four. 4. Take it with Draw and confirm reset.	A player without a stacking card takes the penalty automatically. A player with a response may stack or draw. +4 is permitted on +2.

#	Priority	Name	How to Demo	Notes
10	Must	Select the Wild colour	1. Play Wild and select blue. 2. Confirm blue becomes the active colour. 3. Reload and cancel the dialog; confirm no move is committed.	ColorPickerDialog; bot also selects a regular colour.
11	Must	Apply Wild Draw Four	1. Play +4 and choose a colour. 2. Confirm four draws and a skipped turn without a response. 3. Reload and stack another +4. 4. Confirm +2 cannot answer an active +4 penalty.	Adds four to the penalty. Only +4 can answer +4. No challenge or restriction based on other cards in hand.
12	Must	Announce UNO and the winner	1. Load a two-card hand and play one. 2. Confirm UNO feedback. 3. Play the final card legally. 4. Confirm the winner and no further gameplay.	Automatic UNO at one card, without a manual-call penalty. Empty hand ends the game; GUI stops bot scheduling.
13	Must	Recycle discards safely	1. Load an empty draw deck with several discards. 2. Draw and confirm the top discard remains. 3. Check card conservation. 4. Repeat with only one discard and confirm no crash.	Recycled wild cards return to SPECIAL; no-card drawing advances the turn.

User Interfaces

#	Priority	Name	How to Demo	Notes
14	Must	Display a usable Swing board	1. Start a game. 2. Locate hand, top discard, opponent count, turn, and action buttons. 3. Load a large hand and scroll horizontally.	MainFrame, GamePanel, CardComponent; readable coloured buttons.
15	Must	Click cards and receive feedback	1. Confirm legal cards have orange borders. 2. Click a legal card and confirm the discard changes. 3. Click an illegal card and confirm a rejection message.	Illegal cards remain clickable on the human turn to provide feedback.
16	Must	Navigate the menu and start fresh	1. Confirm Start Game, Load Game, and Exit. 2. Start a game, return to Menu, then Start Game again. 3. Confirm fresh state. 4. Exit through the menu.	Use Menu → Start Game to begin a fresh game.
17	Must	Schedule basic bot turns	1. Play a regular card. 2. Observe the brief delay. 3. Confirm the window repaints. 4. Return to Menu during the delay and confirm no pending bot move runs.	TurnScheduler: one-shot 800 ms Swing timer; search responsiveness is covered separately.
18	Should	Provide shared terminal gameplay	1. Run java -cp out uno.Main. 2. Start a game. 3. Use a card number, d, h, s name, l name, and q. 4. Compare rules/save behaviour with the GUI.	TuiView uses the same controller and engine.
19	Should	Explain controls and selected rules	1. Open the README. 2. Follow the controls and rules. 3. Compare the descriptions with gameplay.	Document the simplified deck, drawing, stacking, two-player Reverse, and automatic UNO behaviour.

Advanced Topic 1: Persistence and Compression

#	Priority	Name	How to Demo	Notes
20	Must	Capture and rebuild complete state	1. Prepare known hands, piles, turn, direction, active wild colour, and penalty. 2. Convert through SaveState. 3. Compare every field and pile order after reconstruction.	fromGameState() / toGameState(); do not create a new shuffled deck when restoring.
21	Must	Serialize versioned game data	1. Serialize a fixture. 2. Inspect UTF-8 fields and COLOR:VALUE cards. 3. Deserialize and compare. 4. Try an unsupported version and confirm a controlled error.	GameSerializer: ordered custom text format, version 1.
22	Must	Compress and decompress bytes	1. Serialize a game. 2. Compress, decompress, and compare exact bytes. 3. Report both sizes. 4. Repeat for empty, repeated, and nonrepetitive input.	Custom LZ-style references: distance 1–4095, length 3–18, UZ header and flags. Smaller output is not guaranteed.
23	Must	Save, list, and load games	1. Save as demo_save. 2. Confirm saves/demo_save.unosave. 3. Return to Menu and select it through Load Game. 4. Compare hands, piles, turn, colour, direction, and penalty.	SaveManager; sorted list filters .unosave files. Terminal also supports saving/loading.
24	Must	Handle save errors	1. Try a save name containing /. 2. Load a missing save via the terminal. 3. Try wrong-signature, truncated, invalid-reference, and invalid-field fixtures. 4. Confirm useful errors and continued interface usability.	SaveException; saving a finished game is rejected. Checks do not constitute a checksum.

Advanced Topic 2: Minimax Computer Bot

#	Priority	Name	How to Demo	Notes
25	Must	Retain a baseline bot and strategy interface	1. Use BasicBot. 2. Confirm first-legal-card play, drawing when blocked, and most-frequent-colour selection. 3. Show the strategy constructor/setter.	BotStrategy separates decision-making from game execution.
26	Must	Define Minimax assumptions and evaluation	1. Present hidden-card/draw assumptions. 2. Explain terminal scores and the cutoff heuristic. 3. Evaluate two fixed positions and justify their ordering.	Document full-information advantage or sampling approximation; choose and justify heuristic weights.
27	Must	Simulate moves without changing the live game	1. Copy a fixture. 2. Simulate regular, wild, Skip/Reverse, draw, and penalty moves. 3. Compare against engine behaviour. 4. Confirm original hands, piles, and listeners remain unchanged.	Independent mutable state; include refill behaviour under the chosen information model.
28	Must	Choose moves with depth-limited Minimax	1. Confirm an immediate winning move is selected. 2. Use a fixture with an avoidable losing reply within the search horizon and confirm avoidance. 3. Inspect MAX/MIN roles in a search trace.	Include wild-colour branches, terminal checks, cutoff scores, deterministic ties, and repeated turns from action cards.

#	Priority	Name	How to Demo	Notes
29	Must	Integrate the searched action	1. Start a Minimax game. 2. Exercise regular, wild, drawing, and penalty turns. 3. Confirm legal actions and that the chosen wild colour matches the evaluated action.	Keep card selection and wild-colour selection consistent with the evaluated move.
30	Must	Bound search and keep Swing responsive	1. Run a branching-heavy fixture. 2. Measure against a documented search budget. 3. Confirm repaint/input responsiveness. 4. Leave/load another game and confirm stale results cannot alter it.	Depth/time/node limit, legal fallback, background search and safe UI updates. Alpha-beta pruning is optional.
31	Must	Compare and explain bot decisions	1. Run both bots on fixed fixtures. 2. Record choices, depth, evaluations, node counts, and time. 3. Explain a choice changed by lookahead. 4. State limitations.	Controlled draw order/randomness for reproducibility; no guaranteed win-rate improvement claimed.

Verification and Final Delivery

#	Priority	Name	How to Demo	Notes
32	Must	Record repeatable verification	1. Run documented checks for dealing, matching, actions, wins, recycling, save round trips, malformed files, and bot decisions. 2. Record expected/actual results. 3. Fix failures and rerun affected checks.	Use automated and/or reproducible manual checks with documented expected results.
33	Must	Complete README instructions	1. Follow the README from a clean directory. 2. Build and launch GUI/terminal modes. 3. Verify Java version, controls, rules, bot assumptions, and save location are explained.	Include Java requirements and Windows and macOS/ Linux build instructions.
34	Must	Verify portable, timely startup	1. Build/run on Windows, macOS, and Linux using the README. 2. Confirm the Swing application is usable within one minute of launch. 3. Record platform/JDK results.	Assignment Section 5.3: Windows, macOS, Linux, and usability within one minute of launch.
35	Must	Prepare the advanced-topic reflection	1. Open a maximum-two-page PDF covering both topics. 2. Check motivation, research/learning, application, challenges, decisions, and impact. 3. Explain actual implementation and cite research sources used.	Assignment Section 4; describe limitations and actual contributions.
36	Must	Package the final submission	1. Create one ZIP with runnable project/full source, README, final backlog, and reflection PDF. 2. Extract into a clean directory. 3. Build and complete a game/ save/load smoke test.	Assignment Section 5.3: Canvas deadline 23 October 2026 at 23:59. Include the final versions of all documents.
37	Could	Select bot difficulty in the UI	1. Choose Basic or Minimax before starting. 2. Start each mode and confirm the strategy used.	Optional UI selection between the basic and Minimax strategies.

4. Definition of Done

A backlog item is complete when:

1. Its implementation/deliverable is finished and matches the selected rules.
2. Its demo steps produce the expected results and repeatable evidence is recorded.
3. Relevant automated or documented manual checks pass, including affected regression checks.
4. Integration does not break existing gameplay, persistence, or interface behaviour.
5. Code has clear responsibilities, readable naming, and appropriate documentation.
6. For advanced-topic work, the team can explain the concepts, design decisions, and limitations.

For final delivery, complete all Must items, record deferred Should/Could items, verify platform/startup requirements, and include all required documents.